

NOVA

IMS

Information
Management
School

MDSAA

Master's Degree Program in
Data Science and Advanced Analytics

Deep Learning

Rare Species Image Classification

Diogo, Carvalho, number: 20240694

Ricardo, Pereira, number: 20240745

Rúben, Marques, number: 20240352

Tomás, Carvalho, number: 20240938

Group 03

NOVA Information Management School
Instituto Superior de Estatística e Gestão de Informação

Universidade Nova de Lisboa

May 1, 2025

INDEX

1. Introduction	2
2. Dataset and Methodology	2
2.1. Dataset and preprocessing	2
2.2. Methodology	3
2.2.1. Metrics	3
2.2.2. Callbaks	3
2.2.3. Model Optimization	3
2.2.4. Transfer learning	4
2.2.5 Experimental Setup	4
3. Models	4
3.1. Results and discussion	5
4. Conclusion	6
5. Bibliogaphy	6

1. INTRODUCTION

The classification of rare species based on visual data is a complex and meaningful challenge in the field of deep learning and biodiversity conservation. In this project, our objective was to build a deep learning model capable of accurately predicting the family of a given species from its image. The dataset we used is sourced from the *BioCLIP: A Vision Foundation Model for the Tree of Life* [1], comprising curated images of rare species from the *Encyclopedia of Life (EOL)*, in which each image is associated with metadata, including its kingdom, phylum, and family. For our model, the prediction target is the family label, making this a multiclass image classification problem with a lot of classes and significant class imbalance. To address these challenges, we developed a complete deep learning pipeline, including data preprocessing, data augmentation to improve generalization, transfer learning techniques, training, evaluation and results analysis.

2. DATASET AND METHODOLOGY

2.1. DATASET AND PREPROCESSING

The dataset used in this project is sourced from the **BioCLIP** benchmark introduced by Stevens et al. [1], which aggregates high-quality images of rare species from the **Encyclopedia of Life (EOL)**. Each image is associated with taxonomic metadata, including kingdom, phylum, and family labels. For this study, we focus exclusively on predicting the **family** label, formulating the task as a multi-class image classification problem.

The dataset has 202 unique families, with a high degree of class imbalance, some families are represented by hundreds of images, while others have only a few examples. This imbalance reflects the real-world rarity of certain species and poses a significant challenge for supervised learning algorithms.

To prepare the data for model development we did some preprocessing. We began by encoding the target feature ("family") into integer values, so it's compatible with our deep learning models. Then we proceed with cleaning the dataset to improve data quality. For that, we used a trained model called EfficientNetB0 to automatically detect and filter images that are unlikely to represent a species. With this, we were able to discard 55 images from our dataset, which is less than 5% from our dataset like we wanted.

After that, we split the dataset, while keeping the same distribution of classes (target), into:

- Training set (70%), used to train the model
- Validation set (20%), used for model evaluation during training
- Test set (10%), used for final evaluation on unseen data

Finally, we used TensorFlow to load the images as tensors, which made training more efficient on the GPU and reduced memory overload. Images were then decoded, resized, normalized and one-hot encoded before batching. PreFetching and parallel processing were applied to speed up data loading during training

2.2. METHODOLOGY

2.2.1. Metrics

In order to ensure a fair evaluation across all the 202 classes, the group used macro-averaging F1 score as the main selection metric. Unlike other metrics, F1 score is better suited for our unbalanced dataset, since it averages F1 scores for each class, treating them all equally.

2.2.2. Callbaks

To improve training efficiency and in order to prevent overfitting, we included two key callbacks: EarlyStopping and ReduceLROnPlateau.

EarlyStopping monitors validation performance and stops training when there is no longer a significant improvement, eliminating “wasted” epochs and reducing overfitting. We’ve set it up with a patience parameter, so the model gets enough time to stabilize from short-term variations.

ReduceLROnPlateau dynamically reduces the learning rate when the validation performance plateaus, which helps the optimizer to make smaller and more precise steps, possibly leading to a better convergence. By reducing the learning rate when needed, we maintained a balanced training speed and stability

2.2.3. Model Optimization

To improve generalization and prevent overfitting, a set of image augmentation techniques was applied during training. These include random brightness, flips, rotation, and contrast. Because these transformations are applied dynamically each time an image is fed into the model, the input varies continuously, increasing dataset variability without needing additional samples.

For optimization, both SGD and Adam, two widely used optimizers, were used because of their efficiency and reliability. In our case, Adam showed a better performance.

To improve generalization and stabilize the training process, we applied regularization techniques and bias initialization tailored to the dataset characteristics.

We used dropout after some layers, with rates between 0.5 and 0.3. This is a common method to prevent overfitting, as it randomly turns off a portion of the neurons during training. This forces the model to rely on different paths through the network and helps it learn more robust features instead of memorizing the training data.

Since our dataset is imbalanced, we also initialized the bias of the output layer using the class distribution. We computed the log of the class frequencies ($\log(\text{count} / \text{total})$) and used that as the initial bias. This helps the model make more reasonable predictions from the start, instead of leaning too much toward the majority classes before learning anything useful.

2.2.4. Transfer learning

Transfer learning uses a model pre-trained on a large dataset like ImageNet and adapts it to a new task. It's especially useful when working with smaller datasets, as the model already knows how to extract general visual features. This leads to faster training and often better results.

In our project, we applied transfer learning in two different ways. In one approach, we kept the entire pre-trained base model frozen and trained only the additional layers we added on top. In the other, we made the entire pre-trained model trainable from the start, allowing all layers to be updated during training to better adapt to our task.

Another common technique that could have been used involves freezing the pre-trained base model for a few initial epochs to stabilize training, and then unfreezing some or all of its layers to fine-tune with a lower learning rate. This allows the model to gradually adapt to the new task while reducing the risk of overfitting early on.

2.2.5 Experimental Setup

We used Keras to implement the entire deep learning pipeline, including data preprocessing, model definition, training, and evaluation. All model components including layers, loss functions, metrics, optimizers, and callbacks were constructed using Keras. Pre-trained models like EfficientNetB0, EfficientNetV2S, and ResNet50 were used for transfer learning, and for image preprocessing we used TensorFlow due to its increased stability and compatibility. The augmentation relied on layers such as RandomFlip, RandomBrightness, RandomRotation, and RandomContrast, which are from Keras. Finally, supporting libraries included NumPy, Pandas, and Matplotlib for data handling and visualization, and scikit-learn for label encoding and evaluation metrics.

3. MODELS

In this section, we describe the different models developed for the classification task, including custom CNNs and transfer learning approaches. We experimented with various architectures, optimizers, and training strategies to assess their effectiveness.

Custom CNN 1 (CNN 1) - We built this model from scratch to test our own architecture using the SGD optimizer. It has seven convolutional blocks with increasing filters, batch normalization, pooling, dropout, and L2 regularization. We used GlobalAveragePooling instead of Flatten to reduce parameters and help prevent overfitting. After that, two dense layers (1024 and 512 units) lead to the final softmax output. Training used SGD with a learning rate of 0.01.

Custom CNN 2(CNN 2) - Same architecture as CNN 1 but using Adam instead of SGD to compare optimizers. All layers remained unchanged. From this point on, all models used Adam.

EfficientNetV2S (EF 1) - Here we used a pre-trained EfficientNetV2S model, fully trainable, to test transfer learning. We added data augmentation, a global pooling layer, two dense layers, and dropout to prevent overfit. This setup let us fine-tune the base model to better fit our problem.

EfficientNetV2S with Bias Initialization (EF 2) - Same as the previous model, but with a bias initializer in the final dense layer to reflect class distribution. This approach aims to help the model deal better with imbalanced classes during early training.

ResNet50 (RN50) - In this approach, we applied transfer learning by using the pretrained ResNet50 as a fixed feature extractor, keeping the entire base model frozen (trainable = False). Only the custom classification head, added on top of the base model, was trained to adapt it to our problem. This head included global pooling, dense layers, dropout, and a bias initializer to address class imbalance.

3.1. RESULTS AND DISCUSSION

	CNN 1	CNN 2	EF 1	EF 2	RN50
F1 - Train	0.2815	0.3159	0.7725	0.8487	0.0742
F1 – Val	0.1712	0.2028	0.6079	0.6090	0.0577

Table 1 – Models results

To guarantee a fair evaluation across all our models, the evaluation was performed on the validation data without augmentation, ensuring that each model was evaluated on the same set of images. During training, data augmentation was applied, so each epoch presented different, randomly altered versions of the input images. This introduced variability that can cause model performance to fluctuate slightly between runs. Additionally, the introduction of the bias in the output layer helped improve early training stability and calibration. As this bias had a consistently positive effect during the training, it was applied to consequent models.

Based on the results, although both EF 1 and EF 2 showed similar validation performance, EF 1 was selected as the final model due to its slightly lower degree of overfitting, despite having a marginally lower F1-score. The model was able to adapt effectively to our specific task, as it had already been pre-trained on large-scale image data. This helped it generalize better to our test set. The selected model was then used to predict classes on unseen data (the test data) and achieved an F1-score of 0.5207. While it correctly predicted some classes with fewer samples, it's important to note that the model still struggled with these types of classes, with 12 classes receiving an F1-score of 0.

For future work, several improvements could be explored across the evaluated models and new models. For Models CNN 1 and CNN 2, which showed a decent performance, adding more layers and increasing the model complexity could help enhance their capacity. In contrast, the model RN50 underperformed, suggesting the need for a different strategy such as selectively unfreezing only certain layers or the totality, rather than keeping the model frozen. Additionally, further hyperparameter optimization using Keras Tuner, testing more efficient architectures recommended by the Keras API, and exploring alternative techniques or better implementations of existing methods could lead to improved results. It is also important to address class imbalance, as the models still

struggle with underrepresented classes, therefore, gathering more samples for these classes is essential to improve overall predictive performance.

4. CONCLUSION

In this project, we developed and evaluated several deep learning models for the classification of rare species by family using a highly imbalanced dataset. We implemented a complete pipeline including data preprocessing, augmentation, bias initialization, regularization, and transfer learning. Among the tested models, EfficientNetV2S with full fine-tuning and data augmentation achieved the best overall performance without overfitting. Despite some strong results, the models still struggled with underrepresented classes, highlighting the need for more balanced data. Overall, this work demonstrates the effectiveness of deep learning for image classification and lays the groundwork for future improvements.

5. BIBLIOGRAPHY

[1]: Stevens, S., Wu, J., Thompson, M. J., Campolongo, E. G., Song, C. H., Carlyn, D. E., ... & Su, Y. (2024). Bioclip: A vision foundation model for the tree of life. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 19412-19424)